Yardflow — Ficha técnica

Sistema de monitoramento de caminhões no pátio da empresa, da chegada na portaria até a saída.

1. Objetivo

Acompanhar a localização de caminhões dentro do pátio em tempo real e notificar o motorista quando ele for chamado para a doca, sem depender de rádio ou ligação.

2. Fluxo do usuário

1. Motorista chega à portaria e apresenta a nota fiscal ou agenda de entrega.
2. Operador da portaria registra a chegada no sistema (placa, motorista, nota/agenda) e exibe o QR Code fixo da empresa.
3. Motorista escaneia o QR Code, abre o PWA no celular dele e informa a placa ou número da nota para vincular ao registro.
4. PWA solicita permissão de localização. Enquanto autorizado, envia coordenadas ao servidor periodicamente.
5. Operador da doca acompanha o painel interno com todos os caminhões no pátio e, quando necessário, clica em "chamar para a doca X".
6. Motorista recebe notificação push sonora e visual no celular, mesmo com a tela bloqueada.
7. Ao sair da área do pátio (geofence) ou por confirmação manual do operador, o rastreamento é encerrado e o tempo de permanência é registrado.

3. Perfis de usuário

Perfil	**Acesso**
Motorista	PWA público, sem login — vinculado por placa/nota
Operador de portaria	Painel admin — registra chegadas, gera QR de acesso
Operador de doca	Painel admin — visualiza fila, chama caminhões, confirma saídas
Gestor	Painel admin — relatórios e histórico de permanência

4. Funcionalidades por módulo

Portaria (admin)

- Cadastro de chegada: placa, nome do motorista, nota fiscal/agenda
- Geração de vínculo para o motorista acessar o PWA
- Status inicial: `aguardando`

PWA do motorista

- Instalável via "Adicionar à tela inicial" (manifest.json + service worker)
- Vinculação por placa/nota ao abrir
- Solicitação de permissão de geolocalização (`navigator.geolocation`)
- Envio periódico de coordenadas ao servidor (intervalo configurável, sugestão 15–30s)
- Inscrição em push notification (`pushManager.subscribe`)
- Exibição de tempo no pátio e status atual
- Tela de notificação de chamada com botão "confirmar chegada na doca"
- Aviso de instalação obrigatória no iOS para habilitar push

Painel da doca (admin)

- Lista em tempo real dos caminhões no pátio, com tempo decorrido
- Botão "chamar para a doca X" por caminhão
- Alertas de "possível saída" quando o caminhão ultrapassa o raio do geofence, com botão de confirmação manual
- Métricas no topo: total no pátio, total na doca, tempo médio de espera

Relatórios (admin)

- Histórico de visitas: entrada, saída, tempo de permanência
- Filtro por período
- Exportação (CSV/planilha)

5. Regras de negócio definidas

- **Identificação do motorista**: QR Code fixo da empresa (não gerado por viagem); vínculo feito por placa ou número da nota digitado no PWA.
- **Chamada para a doca**: sempre manual, feita pelo operador no painel — não há fila automática.
- **Encerramento do rastreamento**: geofence sugere a saída automaticamente (`possível saída`), mas o registro só é fechado com confirmação manual do operador. Evita falso encerramento por imprecisão de GPS.

- **Notificação**: precisa funcionar com o app em segundo plano/tela bloqueada — exige push notification real (Web Push + service worker), não apenas alerta na aba aberta.
- **Limitação conhecida (iOS)**: push só funciona se o motorista instalar o PWA na tela inicial; no Android funciona mesmo sem instalar, mas a instalação melhora a confiabilidade do rastreamento.

6. Arquitetura técnica

Mesmo padrão usado em ClickReserva e ClickCantina:

- **Monorepo**: TypeScript, pnpm workspaces
- **Backend**: Node.js + Express + Drizzle ORM
- **Banco de dados**: Supabase PostgreSQL (usar Session pooler, não Direct connection, no Render)
- **Frontend admin**: React + Vite + TanStack Query + shadcn/Tailwind
- **PWA do motorista**: React + Vite, manifest.json, service worker (cache + push)
- **Notificações push**: Web Push API com chaves VAPID, biblioteca `web-push` no backend
- **Geofence**: cálculo de distância via fórmula de Haversine, ponto central e raio configuráveis no painel admin. Ponto central: **-25.453996734949122, -49.362939914073124** · Raio: **1.200 metros**
- **Intervalo de envio de localização**: 20 segundos (equilíbrio entre precisão do rastreamento e consumo de bateria do motorista)
- **Deploy**: Render (API + dois frontends), keep-alive via GitHub Actions/UptimeRobot se necessário

Estrutura de pastas sugerida

Yardflow/

apps/

api-server/ (backend Express + Drizzle)

admin/ (painel portaria + doca + relatórios)

driver-pwa/ (PWA do motorista)

packages/

shared/ (tipos e schemas compartilhados)

7. Identidade visual

- Estilo industrial/logística, com paleta de grafite escuro + âmbar (alerta/atenção)
- Estados de status com cores semânticas: aguardando (âmbar), no pátio (verde), chamado (âmbar), possível saída (alerta), finalizado (neutro)

8. Em aberto / a definir com o cliente final (empresa)

- Necessidade de múltiplos pátios/filiais no mesmo sistema
- Layout de exportação de relatório (CSV simples ou planilha formatada)

9. Nome do projeto

Yardflow — repositório sugerido: `Yardflow2026` (seguindo padrão `ClickReserva2026`)